

R o b o t i c V i s i o n

M R E 4 9 2

I n d i v i d u a l P r o j e c t

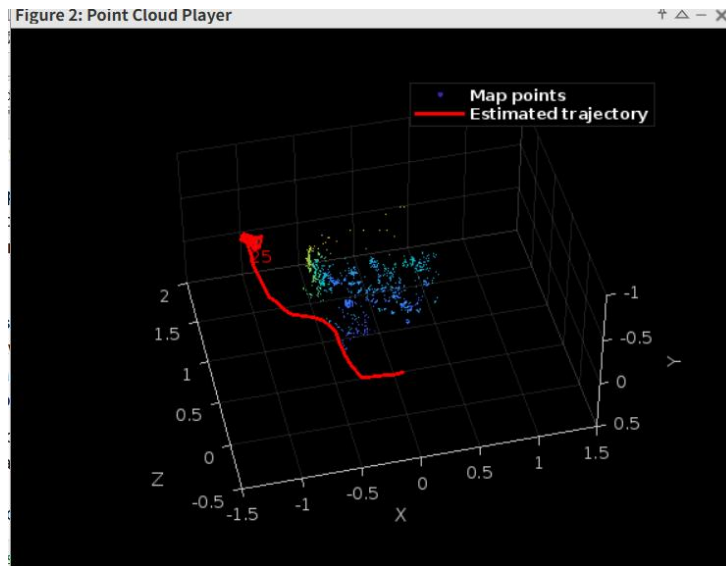
P r o f e s s o r : M i n g s h a o Z h a n g

S t u d e n t : L i n g c h u a n w u

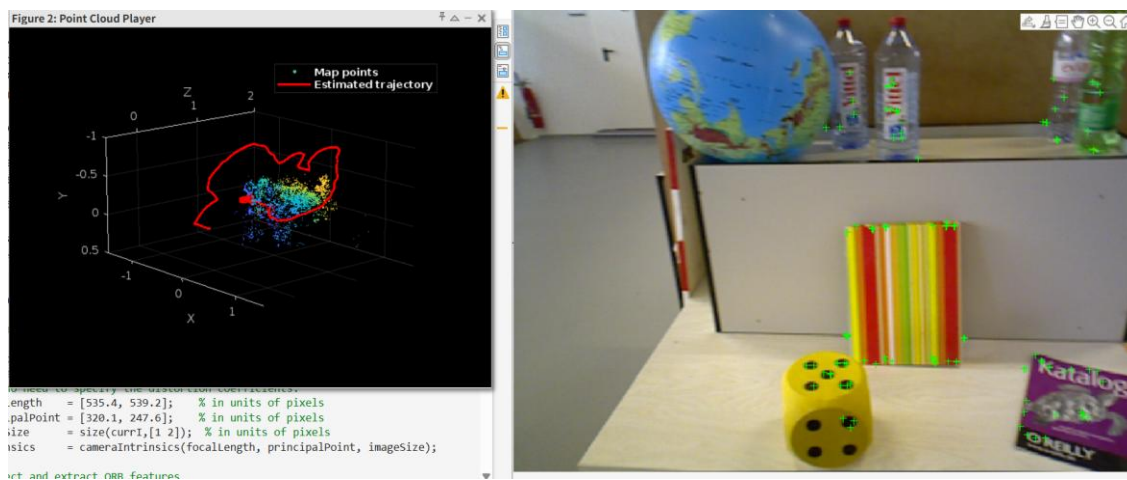
D a t e : 5 / 3 / 2 0 2 5

Part 1:

In this project, MATLAB demonstrates how to use a monocular camera image sequence to complete Visual-SLAM (Visual Simultaneous Localization and Mapping) by implementing the ORB-SLAM process. The dataset used is the indoor office scene image sequence provided by the TUM RGB-D Benchmark, which includes real trajectories and camera intrinsic parameters. The entire implementation process mainly consists of six stages: Firstly, the map initialization is completed through ORB feature extraction and image matching, and the initial map points and camera poses are obtained by triangulation using the matching points between two frames; then, tracking is performed in each frame, and new keyframes are inserted if necessary based on the matching features, and the map points and trajectory information are updated; in the local mapping stage, the local area is optimized to improve accuracy; subsequently, loop closure detection is carried out, and image retrieval is used to find similar candidate frames in the historical frames, and geometric transformation is used to determine whether a loop closure is formed. If the loop closure is valid, it is added to the graph structure and graph optimization is performed; finally, the visualization output includes a 3D comparison diagram of the estimated trajectory, the loop closure optimized trajectory, and the real trajectory. This system also supports video input and achieves autonomous expansion through strategies such as extracting frames one by one and adjusting the frame rate. The entire process reflects the complete process of a feature-based SLAM system from image preprocessing to loop closure optimization.

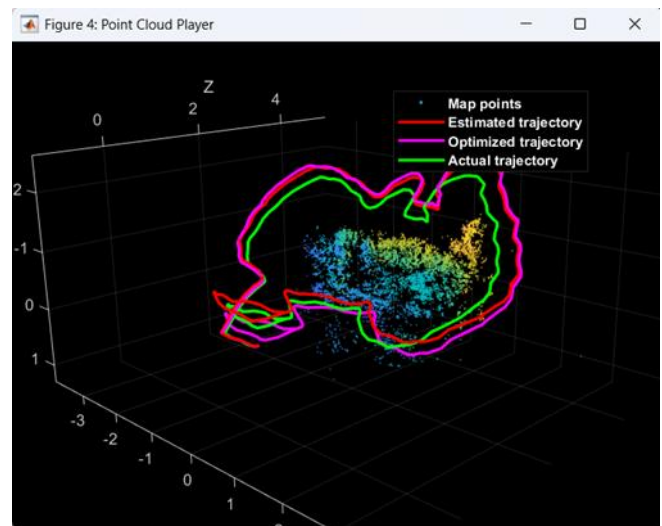
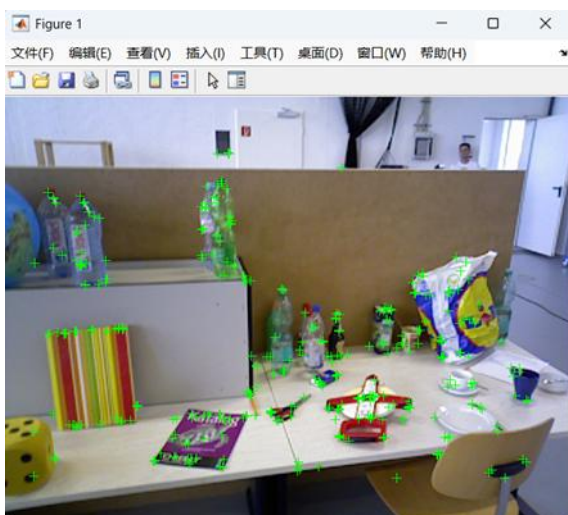


This is the simulation diagram of the first attempt of the camera to make a 90° turn. It is obvious that this is a key frame.



This is the image generated by the camera as it moved around the table. Due to the different shooting angles, we can see that the red line is in an irregular shape on the simulation diagram, which clearly shows the position at the time of shooting.

The image results show that the left window displays the ORB feature point matching situation in the current image frame. The green marks indicate the successfully detected feature points that have been paired, indicating that the system has completed effective feature extraction and initial positioning in the current frame. The right window is the 3D point cloud and camera trajectory diagram, where the blue points represent the constructed map points, the red line is the original estimated trajectory, the green line is the trajectory after loop closure optimization, and the pink line is the real trajectory provided in the dataset. The optimized trajectory is significantly closer to the real trajectory, indicating that the system's loop closure detection and graph optimization functions are operating normally and effectively improving the camera positioning accuracy. The overall results verify the accuracy of ORB-SLAM in pose estimation and map construction in indoor environments.



Part 2:

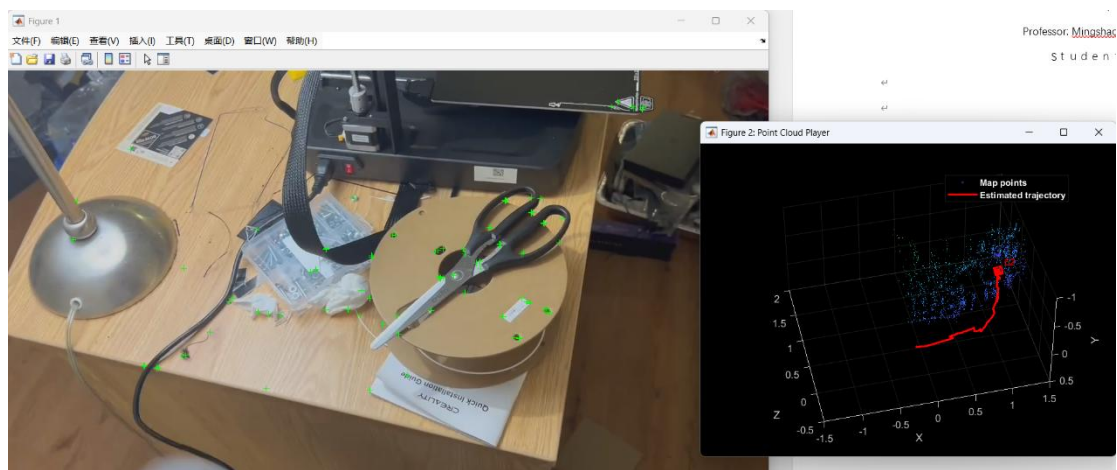
After finishing the first part, I attempted to record a video myself to simulate the trajectory. I did a circular shooting around a table in my dormitory, but due to the space limitation, the moving route was not a complete circle. I used my mobile phone to shoot a video and adjusted the camera parameters to

`focalLength = [535.4, 539.2]; % in units of pixels`

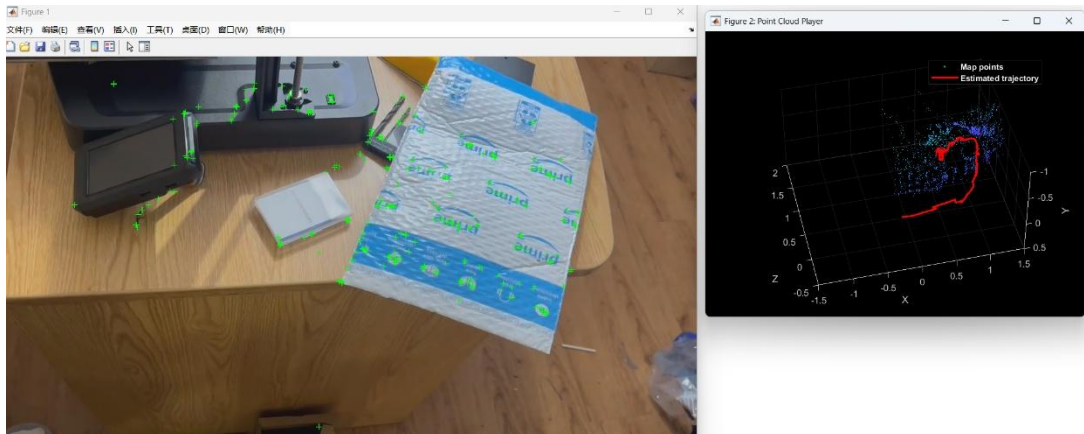
`principalPoint = [320.1, 247.6]; % in units of pixels`

`imageSize = size (currI, [1 2]); % in units of pixels`

After learning the first part, I only needed to change the address of the header file to upload the video. What troubled me was that the address file in the example was in picture format, while I only had the video format. I tried to decode the video container into static images frame by frame and save them to the disk. After many attempts, I found this method to be the most reliable. I also tried to put the data into the cell, but when re-uploading the video, I had to manually clear the files in the cell, which was very troublesome.

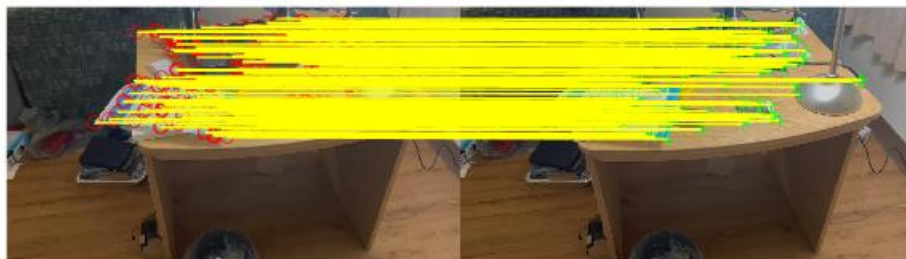


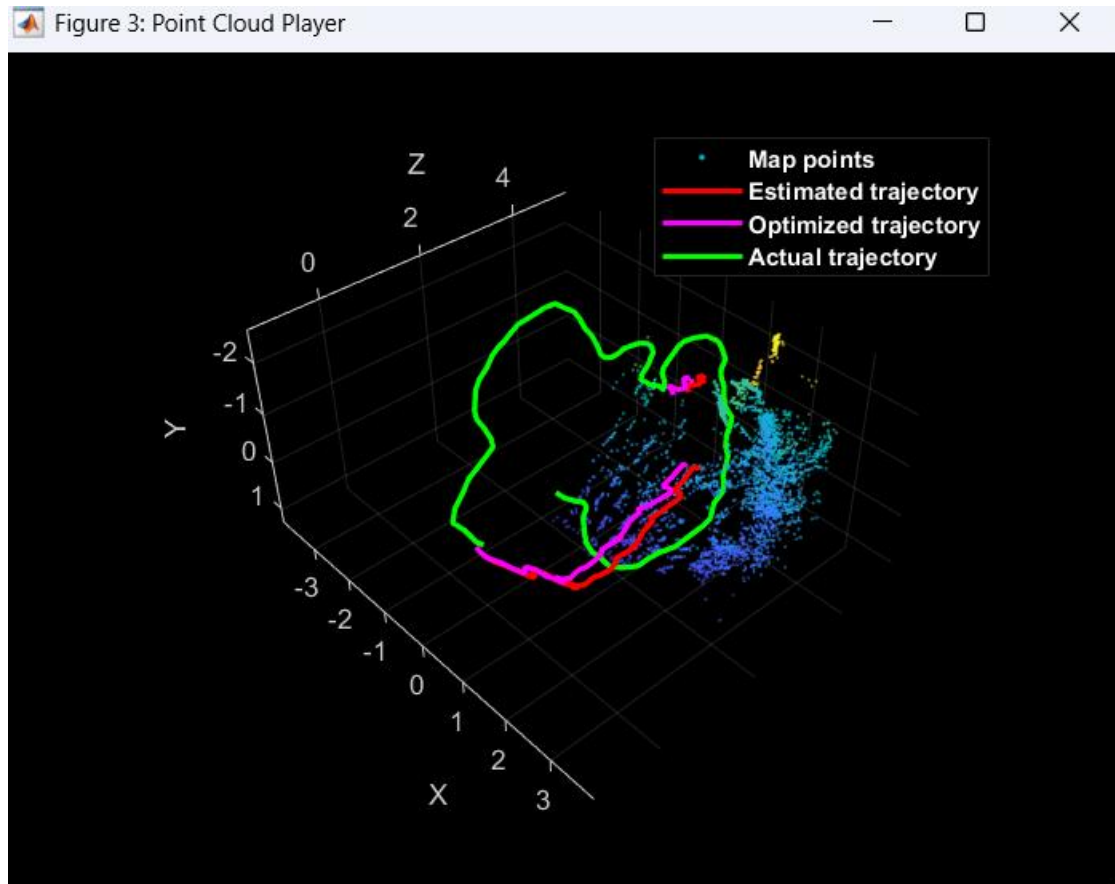
These are the keyframes I formed during the simulation.



Due to local restrictions, the simulated results are not very precise. A circle has not been formed.

```
if isMapInitialized
    close(himage.Parent.Parent); % Close the previous figure
    % Show matched features
    hfeature = showMatchedFeatures(firstI, currI, prePoints(indexPairs(:,1)), ...
        currPoints(indexPairs(:, 2)), "Montage");
else
    error('Unable to initialize the map.')
end
```





The blue-green small dots represent the 3D point cloud generated through initialization and subsequent keyframe triangulation. As can be seen from the figure, these points are roughly distributed in a "semi-circular" shape, corresponding to the area of the table and chair that the camera circled around. The areas with higher point cloud density reflect the parts where the camera moved more and had richer feature information.

The red dotted line represents the camera motion trajectory accumulated by the SLAM algorithm without any global optimization. It can be observed from the figure that as the running time increases, a significant drift phenomenon gradually appears in the trajectory - when the curve extends to the right, the path begins to deviate from the central area of the blue-green point cloud.

The magenta dotted line shows the optimized trajectory after loop closure detection and global bundle adjustment. Compared with the red trajectory, this trajectory fits the distribution trend of the point cloud more closely and effectively corrects the early drift problem, especially aligning more precisely with the actual map structure at the turns and twists of the path.